Práctica 5. Datos Anómalos

Análisis Exploratorio de Datos, Máster en Ciencia de Datos - UV

Contents

Introducción y objetivos **1**

Detección de inconsistencias en los datos **2**
- Ejercicio 1: Inspección Visual . . . 2
- Ejercicio 2: Valores especiales . . . 4
- Ejercicio 3: Valores perdidos y observaciones completas . . . 4

Detección de outliers univariante **5**
- Ejercicio 4: Estadísticos básicos . . . 5
- Ejercicio 5: Métodos de detección de outliers . . . 6
- Ejercicio 6: Detección de outliers . . . 7
- Ejercicio 7: Comparativa de los métodos de detección de outliers . . . 8
- Ejercicio 8: Representación gráfica . . . 9

Imputación de datos **10**
- Ejercicio 9: Imputación de datos perdidos con estadísticos básicos . . . 10
- Ejercicio 10: Representación gráfica . . . 11


```
library(pacman)
packages = c("knitr", "ggplot2","tidyr","dplyr","readr")
pacman::p_load(char=packages)
```

Introducción y objetivos

El objetivo de esta práctica es comprobar si los conjuntos de datos con los que vamos a trabajar son consistentes y, por tanto, pueden ser empleados para realizar un análisis sin errores. Las observaciones contenidas en los datos pueden presentar diferentes tipos de inconsistencias, como valores perdidos (missing values), valores especiales y valores anómalos (outliers).

Para evitar estas inconsistencias, estos datos deberán ser eliminados, corregidos o imputados. Siendo las etapas que pueden seguirse para obtener datos consistentes:

1. Detección de las inconsistencias.
2. Detectar el campo o campos que la generan y las dependencias entre ellos.
3. Corrección de errores.

Vamos a trabajar con el conjunto de datos *dirty_iris*, que ya conocemos, que contiene datos perdidos, valores especiales y anomalías de carácter numérico respecto a la distribución de los datos (`dirty_iris.csv`).

```
rm(list=ls())
dirty_iris <- read_csv("data/dirty_iris.csv")
```

Detección de inconsistencias en los datos

Por lo general, no hay un convenio o una representación común para indicar un dato perdido o incluso determinados valores especiales. En los siguientes ejercicios vamos a analizar cómo detectar inconsistencias presentes en los datos.

Ejercicio 1: Inspección Visual

Inspecciona visualmente los valores del conjunto de datos (`View`) y ordena de mayor a menor y de menor a mayor cada variable para descubrir las diferentes inconsistencias que presenta el conjunto `dirty_iris`. Comenta los valores que te llamen la atención.

```
# Medidas con valores faltantes, longitudes negativas y cero
#View(dirty_iris)
summary(dirty_iris)
```

```
  Sepal.Length     Sepal.Width      Petal.Length    Petal.Width    Species
 Min.   : 0.000   Min.   :-3.000   Min.   : 0.00   Min.   :0.1   Length:150
 1st Qu.: 5.100   1st Qu.: 2.800   1st Qu.: 1.60   1st Qu.:0.3   Class :character
 Median : 5.750   Median : 3.000   Median : 4.50   Median :1.3   Mode  :character
 Mean   : 6.559   Mean   : 3.391   Mean   : 4.45   Mean   :Inf
 3rd Qu.: 6.400   3rd Qu.: 3.300   3rd Qu.: 5.10   3rd Qu.:1.8
 Max.   :73.000   Max.   :30.000   Max.   :63.00   Max.   :Inf
 NA's   :10       NA's   :17       NA's   :19      NA's   :12
```

```
for (v in names(dirty_iris)[sapply(dirty_iris, is.numeric)]) {
  cat("\n==========", v, "==========\n")
  cat("Valores más pequeños:\n")
  print(dirty_iris %>% arrange(.data[[v]]) %>% select(all_of(v), Species) %>% head(5))
  cat("Valores más grandes:\n")
  print(dirty_iris %>% arrange(desc(.data[[v]])) %>% select(all_of(v), Species) %>% head(5))
}
```

```

========== Sepal.Length ==========
Valores más pequeños:
# A tibble: 5 x 2
  Sepal.Length Species
         <dbl> <chr>
1          0   setosa
2          4.3 setosa
3          4.4 setosa
4          4.4 setosa
5          4.4 setosa
Valores más grandes:
# A tibble: 5 x 2
  Sepal.Length Species
         <dbl> <chr>
1         73   virginica
2         49   setosa
3          7.9 virginica
4          7.7 virginica
5          7.7 virginica

========== Sepal.Width ==========
Valores más pequeños:
```

```
# A tibble: 5 x 2
  Sepal.Width Species
        <dbl> <chr>
1        -3   versicolor
2         0   setosa
3         2.2 versicolor
4         2.2 virginica
5         2.2 versicolor
Valores más grandes:
# A tibble: 5 x 2
  Sepal.Width Species
        <dbl> <chr>
1        30   setosa
2        29   virginica
3         4.2 setosa
4         4.1 setosa
5         4   setosa

========== Petal.Length ==========
Valores más pequeños:
# A tibble: 5 x 2
  Petal.Length Species
         <dbl> <chr>
1        0     setosa
2        0.82  versicolor
3        0.925 versicolor
4        1.1   setosa
5        1.2   setosa
Valores más grandes:
# A tibble: 5 x 2
  Petal.Length Species
         <dbl> <chr>
1         63   virginica
2         23   versicolor
3         14   setosa
4          6.9 virginica
5          6.7 virginica

========== Petal.Width ==========
Valores más pequeños:
# A tibble: 5 x 2
  Petal.Width Species
        <dbl> <chr>
1         0.1 setosa
2         0.1 setosa
3         0.1 setosa
4         0.1 setosa
5         0.1 setosa
Valores más grandes:
# A tibble: 5 x 2
  Petal.Width Species
        <dbl> <chr>
1       Inf   setosa
2         2.5 virginica
```

```
3         2.5 virginica
4         2.5 virginica
5         2.4 virginica
```

```
# Registros con valores de otra escala (filas completas muy por encima del resto)
dirty_iris %>% filter(Sepal.Length > 10 | Sepal.Width > 10 | Petal.Length > 10)
```

```
# A tibble: 3 x 5
  Sepal.Length Sepal.Width Petal.Length Petal.Width Species
         <dbl>       <dbl>        <dbl>       <dbl> <chr>
1         73          29             63        NA   virginica
2          6.6         2.9           23         1.3 versicolor
3         49          30             14         2   setosa
```

Hay bastantes valores perdidos (`NA`) en las cuatro variables numéricas. `Sepal.Width` tiene un valor **negativo** (-3), imposible para una longitud.

Hay valores **iguales a cero** en `Sepal.Length`, `Sepal.Width` y `Petal.Length`, también imposibles para medidas de una flor.

`Petal.Width` contiene un valor **infinito** (`Inf`).

Hay filas con valores de otra escala (73, 29, 63 en una virginica y 49, 30, 14 en una setosa), que parecen errores de unidades (valores multiplicados por 10).

`Petal.Length` también tiene un 23 en una versicolor, muy alejado del resto, y valores muy pequeños e implausibles para una versicolor (0.82 y 0.925).

Ejercicio 2: Valores especiales

Visualizar los datos puede ser muy revelador, pero en general, especialmente para conjuntos de datos grandes, realizaremos este análisis de forma automática.

Los valores especiales más comunes en R son:

- NA (*not available*): missing value o dato perdido (`is.na`)
- NULL (*empty set*): elemento nulo de longitud 0 (`is.null`)
- NaN (*not a number*): indeterminación con resultado desconocido (`is.nan`)
- Inf (*infinite*): valor numérico de valor infinito (`is.infinite`)

Nota: A parte de estas funciones, `is.numeric` comprueba si la variable es numérica; `is.finite` comprueba la existencia de cualquier valor especial en un vector numérico pero devuelve `FALSE` para variables de texto.

Reemplaza, en todas las columnas numéricas del data frame `dirty_iris`, los valores que sean especiales (usando `is.finite`), negativos, o iguales a cero, por `NA`. Nota: Puedes reemplazar los valores para cada variable del dataframe usando `mutate` con la función `ifelse`.

```
dirty_iris <- dirty_iris %>%
  mutate(across(where(is.numeric),
                ~ ifelse(is.finite(.x) & .x > 0, .x, NA)))

colSums(is.na(dirty_iris))
```

```
Sepal.Length  Sepal.Width Petal.Length  Petal.Width      Species
          11           19           20           13            0
```

Ejercicio 3: Valores perdidos y observaciones completas

- Calcula el número y porcentaje de observaciones completas (muestras sin ningún dato perdido) empleando `complete.cases` (detecta todos los registros sin datos faltantes).

- Emplea después la función `na.omit` para quedarnos solo con las filas con datos completos en un nuevo data frame `clean_iris`.

```
n_completas <- sum(complete.cases(dirty_iris))
pct_completas <- 100 * n_completas / nrow(dirty_iris)
cat("Observaciones completas:", n_completas, "de", nrow(dirty_iris),
    "(", round(pct_completas, 2), "% )\n")
```

```
Observaciones completas: 92 de 150 ( 61.33 % )
```

```
clean_iris <- na.omit(dirty_iris)
dim(clean_iris)
```

```
[1] 92  5
```

Detección de outliers univariante

Un outlier es una observación (o conjunto de observaciones) que no parecen consistentes con el resto de valores de esa variable. Por lo general, esas observaciones divergirán del patrón de comportamiento del resto de datos, es decir, presentarán una desviación respecto de la distribución de los datos.

En esta sección, vamos a centrarnos en métodos de detección univariantes, así que, en los ejercicios, a modo de ejemplo, analizaremos solo la variable `Petal.Length`.

Ejercicio 4: Estadísticos básicos

La presencia de outliers puede producir una gran influencia en caracterizaciones sencillas de los datos, como la media y la desviación típica.

Empleado la función `summarise y across`, calcula la media, la desviación típica, mediana, la *median absolute deviation from the median* (MADM) y el *interquartile range* (IQR) de las variables numéricas. Hazlo para el data frame con observaciones completas `clean_iris` (que no tiene NA pero tiene menos observaciones) y para el data frame con datos faltantes `dirty_iris` (que presenta muchos NA pero tiene todas las observaciones).

- Ten en cuenta que no se puede realizar un análisis estadístico con datos faltantes, pero en la mayoría de los casos se puede optar por no considerar dichos valores al realizar los cálculos (parámetro `na.rm=TRUE`).
- Para obtener el valor MADM, escribe una función `madm(x)` que sea robusta a la presencia de valores NA. Recuerda que el MADM se calcula como $1.4826 \cdot \text{median}(|x - \text{median}(x)|)$.
- Compara los resultados para los dos conjuntos en la variable `Petal.Length`.

```
#MADM=1.4826*median( x_i -median(x) )
madm <- function(x) {
  1.4826 * median(abs(x - median(x, na.rm = TRUE)), na.rm = TRUE)
}

# Estadísticos básicos de todas las variables numéricas
estadisticos <- function(df) {
  df %>%
    summarise(across(where(is.numeric),
                     list(media   = ~ mean(.x, na.rm = TRUE),
                          sd      = ~ sd(.x, na.rm = TRUE),
                          mediana = ~ median(.x, na.rm = TRUE),
                          madm    = ~ madm(.x),
                          iqr     = ~ IQR(.x, na.rm = TRUE)),
                     .names = "{.col}__{.fn}")) %>%
```

```
    pivot_longer(everything(), names_to = c("variable", "estadistico"),
                 names_sep = "__") %>%
    pivot_wider(names_from = estadistico, values_from = value)
}

cat("Observaciones completas (clean_iris):\n")
```

```
Observaciones completas (clean_iris):
```

```
est_clean <- estadisticos(clean_iris)
est_clean
```

```
# A tibble: 4 x 6
  variable     media    sd mediana  madm   iqr
  <chr>        <dbl> <dbl>   <dbl> <dbl> <dbl>
1 Sepal.Length  6.37 4.57      5.8 1.04    1.3
2 Sepal.Width   3.33 2.84      3   0.371   0.5
3 Petal.Length  4.16 2.88      4.5 1.63    3.6
4 Petal.Width   1.22 0.765     1.4 0.890   1.6
```

```
cat("\nDatos con NA, todas las observaciones (dirty_iris):\n")
```

```
Datos con NA, todas las observaciones (dirty_iris):
```

```
est_dirty <- estadisticos(dirty_iris)
est_dirty
```

```
# A tibble: 4 x 6
  variable     media    sd mediana  madm   iqr
  <chr>        <dbl> <dbl>   <dbl> <dbl> <dbl>
1 Sepal.Length  6.61 6.80      5.8 1.04    1.3
2 Sepal.Width   3.47 3.28      3   0.297   0.5
3 Petal.Length  4.48 5.78      4.5 1.63    3.5
4 Petal.Width   1.21 0.765     1.3 1.04    1.5
```

```
# Comparación para Petal.Length
bind_rows(clean_iris = est_clean, dirty_iris = est_dirty, .id = "conjunto") %>%
  filter(variable == "Petal.Length")
```

```
# A tibble: 2 x 7
  conjunto   variable     media    sd mediana  madm   iqr
  <chr>      <chr>        <dbl> <dbl>   <dbl> <dbl> <dbl>
1 clean_iris Petal.Length  4.16  2.88     4.5  1.63   3.6
2 dirty_iris Petal.Length  4.48  5.78     4.5  1.63   3.5
```

La media y la desviación típica de `Petal.Length` cambian bastante entre los dos conjuntos (la media pasa de 4.16 a 4.48 y la desviación típica de 2.88 a 5.78), porque los outliers (63, 23 y 14) están en `dirty_iris` y distorsionan esos estadísticos; en `clean_iris` se han descartado algunos de ellos al eliminar filas con otros `NA`. En cambio la mediana (4.5), el MADM y el IQR apenas varían, ya que son estadísticos robustos frente a outliers.

Ejercicio 5: Métodos de detección de outliers

Escribe una función que detecte los outliers de un vector x para cada uno de los siguientes métodos de detección de outliers univariantes (todas las funciones tienen que ser robustas a la presencia de valores NA).

- **Regla 3 sigma** `reglasigma(x)`: Considera que los datos siguen una distribución gaussiana. Datos típicos caracterizados por la media y la dispersión por la varianza. Valores por encima de 3 desviaciones típicas se consideran outliers.
- **Identificador Hampel** `reglahampel(x)`: Utiliza estimadores robustos y no presupone que los datos siguen una distribución gaussiana. Datos típicos caracterizados por la mediana y la dispersión por la *median absolute deviation from the median* (MADM). Valores por encima de 3 MADM se consideran outliers.
- **Regla boxplot** `reglaboxplot(x)`: A partir de los datos se calcula su diagrama de caja (boxplot) y se definen como datos típicos superiores hasta el cuartil 75%, x_s; datos típicos inferiores hasta el cuartil 25%, x_i; y la dispersión se define como el rango intercuartílico ($IQR = x_s - x_i$). Valores por encima de $x_s + 1.5 \cdot IQR$ o por debajo de $x_i - 1.5 \cdot IQR$ se consideran outliers. Usa la función `quantile` para calcular los cuartiles.
- **Percentiles** `reglapercentil(x)`: Cualquier valor que esté fuera del rango del percentil 5% y 95% se puede considerar como atípico. Usa la función `quantile` para calcular los percentiles.

```r
# Todas las funciones devuelven un vector lógico (TRUE = outlier);
# si x es NA el resultado es NA.

# Regla 3 sigma
reglasigma <- function(x) {
  abs(x - mean(x, na.rm = TRUE)) > 3 * sd(x, na.rm = TRUE)
}

# Identificador Hampel
reglahampel <- function(x) {
  abs(x - median(x, na.rm = TRUE)) > 3 * madm(x)
}

# Regla boxplot
reglaboxplot <- function(x) {
  q <- quantile(x, probs = c(0.25, 0.75), na.rm = TRUE, names = FALSE)
  iqr <- q[2] - q[1]
  x < q[1] - 1.5 * iqr | x > q[2] + 1.5 * iqr
}

# Percentiles
reglapercentil <- function(x) {
  q <- quantile(x, probs = c(0.05, 0.95), na.rm = TRUE, names = FALSE)
  x < q[1] | x > q[2]
}
```

Ejercicio 6: Detección de outliers

Utiliza la función `mutate` y `across` para identificar los outliers en la variable `Petal.Length` con todos los métodos de detección de outliers implementados. Genera una columna nueva para cada método. Nota: podría extenderse la aplicación a todas las variables numéricas usando también `mutate`, `across` y `where`.

```r
iris_out <- dirty_iris %>%
  mutate(across(Petal.Length,
                list(sigma     = reglasigma,
                     hampel    = reglahampel,
                     boxplot   = reglaboxplot,
                     percentil = reglapercentil),
```

```
                .names = "out_{.fn}"))

iris_out %>% select(Petal.Length, starts_with("out_")) %>% head(10)
```

```
# A tibble: 10 x 5
   Petal.Length out_sigma out_hampel out_boxplot out_percentil
          <dbl> <lgl>     <lgl>      <lgl>       <lgl>
 1          4.5 FALSE     FALSE      FALSE       FALSE
 2          6   FALSE     FALSE      FALSE       FALSE
 3          5.4 FALSE     FALSE      FALSE       FALSE
 4          1.6 FALSE     FALSE      FALSE       FALSE
 5          3.5 FALSE     FALSE      FALSE       FALSE
 6         NA   NA        NA         NA          NA
 7          5.3 FALSE     FALSE      FALSE       FALSE
 8          5.1 FALSE     FALSE      FALSE       FALSE
 9          4.1 FALSE     FALSE      FALSE       FALSE
10          1.6 FALSE     FALSE      FALSE       FALSE
```

Ejercicio 7: Comparativa de los métodos de detección de outliers

En los métodos de detección de outliers analizados, examina los resultados comparando:

- Número de outliers detectados por método.
- Los valores etiquetados como outliers (ordenados de menor a mayor).

```
# Número de outliers por método
iris_out %>%
  summarise(across(starts_with("out_"), ~ sum(.x, na.rm = TRUE)))
```

```
# A tibble: 1 x 4
  out_sigma out_hampel out_boxplot out_percentil
      <int>      <int>       <int>         <int>
1         2          3           3            11
```

```
# Valores etiquetados como outliers por cada método (de menor a mayor)
reglas <- grep("^out_", names(iris_out), value = TRUE)
valores_outliers <- lapply(reglas, function(r) {
  sort(iris_out$Petal.Length[iris_out[[r]] %in% TRUE])
})
#iris_out[[r]] %in% TRUE = que elementos de esa columna son true
#iris_out$Petal.Length[iris_out[[r]] %in% TRUE] = devuelve los valores de Petal.Length que tienen true,
names(valores_outliers) <- sub("^out_", "", reglas)
valores_outliers
```

```
$sigma
[1] 23 63

$hampel
[1] 14 23 63

$boxplot
[1] 14 23 63

$percentil
 [1]  0.820  0.925  1.100  1.200  6.600  6.700  6.700  6.900 14.000 23.000 63.000
```

La regla 3 sigma es la más conservadora (2 outliers: 23 y 63), porque los propios outliers inflan la desviación típica y "enmascaran" a otros como el 14. Hampel y boxplot, basadas en estadísticos robustos, detectan los mismos 3 (14, 23 y 63). La regla de percentiles marca siempre un porcentaje fijo de los datos (11 valores), por lo que incluye valores razonables además de los claramente erróneos.

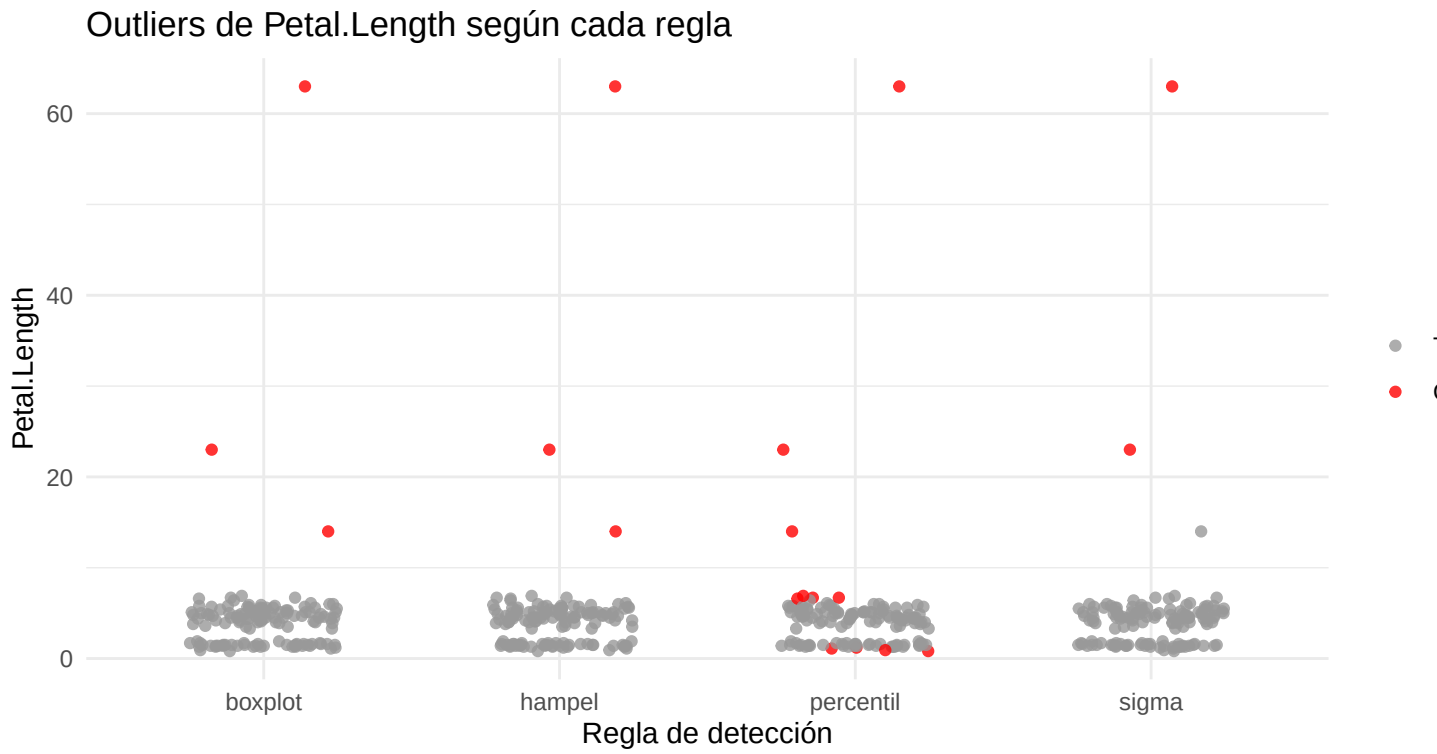
Ejercicio 8: Representación gráfica

Representa gráficamente los datos de `Petal.Length` (distribución de puntos en el eje y) para todas las reglas de detección implementadas (repartidas en el eje x) mostrando los valores identificados como típicos y los identificados como outliers en diferente color. Usa la función `pivot_longer` para poder agrupar por reglas y colorear los outliers fácilmente.

Nota: usa en `geom_point` con la opción `position='jitter'` para mejorar la legibilidad del gráfico.

```
iris_out_long <- iris_out %>%
  select(Petal.Length, starts_with("out_")) %>%
  filter(!is.na(Petal.Length)) %>%
  pivot_longer(starts_with("out_"), names_to = "regla", values_to = "outlier") %>%
  mutate(regla = sub("^out_", "", regla),
         outlier = factor(outlier, levels = c(FALSE, TRUE),
                          labels = c("Típico", "Outlier")))

ggplot(iris_out_long, aes(x = regla, y = Petal.Length, color = outlier)) +
  geom_point(position = position_jitter(width = 0.25, height = 0, seed = 1),
             alpha = 0.8) +
  scale_color_manual(values = c("Típico" = "grey60", "Outlier" = "red")) +
  labs(title = "Outliers de Petal.Length según cada regla",
       x = "Regla de detección", y = "Petal.Length", color = NULL) +
  theme_minimal()
```



Imputación de datos

Como se ha visto, con *missing data* nos referimos a una observación (o conjunto de observaciones) cuyo valor no es conocido.

Vamos a ver a continuación ejemplos de cómo imputar valores realistas a esas observaciones perdidas.

Ejercicio 9: Imputación de datos perdidos con estadísticos básicos

La opción más sencilla es imputar con un valor estimado de los valores conocidos (por ejemplo, con la media, la mediana, o la moda).

Se puede realizar una imputación generalizada (usando todos los valores de la variable) o una imputación a partir de casos similares (la estimación se realiza teniendo en cuenta otra variable):

- Imputa los missing data en `Petal.Length` empleando la media. Nota: Puedes reemplazar los valores vacios usando `mutate` con `ifelse` creando una nueva variable `Petal.Length1`.

```
# "imputado" marca qué valores eran NA originalmente
iris_imp <- dirty_iris %>%
  mutate(imputado = is.na(Petal.Length),
         Petal.Length1 = ifelse(is.na(Petal.Length),
                                mean(Petal.Length, na.rm = TRUE),
                                Petal.Length))

# Valor con el que se ha imputado
mean(dirty_iris$Petal.Length, na.rm = TRUE)
```

```
[1] 4.484192
```

- Imputa los missing data en `Petal.Length` empleando en esta ocasión la media para cada tipo de `Species` (usargroup_by y `mutate`) creando una nueva variable `Petal.Length2`.

```
iris_imp <- iris_imp %>%
  group_by(Species) %>%
  mutate(Petal.Length2 = ifelse(is.na(Petal.Length),
                                mean(Petal.Length, na.rm = TRUE),
                                Petal.Length)) %>%
  ungroup()

# Medias por especie usadas en la imputación
iris_imp %>%
  group_by(Species) %>%
  summarise(media = mean(Petal.Length, na.rm = TRUE), n_NA = sum(imputado))
```

```
# A tibble: 3 x 3
  Species    media  n_NA
  <chr>      <dbl> <int>
1 setosa      1.80    10
2 versicolor  4.55     5
3 virginica   6.80     5
```

```
iris_imp %>% select(Species, Petal.Length, Petal.Length1, Petal.Length2) %>%
  filter(is.na(Petal.Length)) %>% head(8)
```

```
# A tibble: 8 x 4
  Species    Petal.Length Petal.Length1 Petal.Length2
  <chr>             <dbl>         <dbl>         <dbl>
```

```
1 setosa               NA          4.48          1.80
2 setosa               NA          4.48          1.80
3 setosa               NA          4.48          1.80
4 virginica            NA          4.48          6.80
5 setosa               NA          4.48          1.80
6 versicolor           NA          4.48          4.55
7 setosa               NA          4.48          1.80
8 setosa               NA          4.48          1.80
```

Ejercicio 10: Representación gráfica

Vamos a comparar ahora los dos métodos de imputación gráficamente eliminando los outliers para una mejor visualización.

- Recalcula las reglas de identificación de outliers (Ejercicio 6) para la nueva variable `Petal.Length2`.

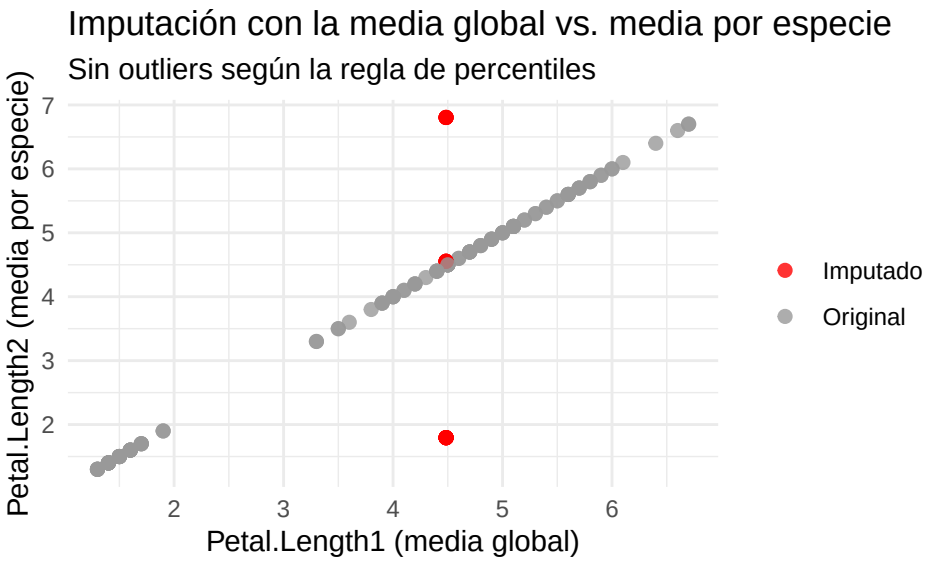
```r
iris_imp <- iris_imp %>%
  mutate(across(Petal.Length2,
                list(sigma     = reglasigma,
                     hampel    = reglahampel,
                     boxplot   = reglaboxplot,
                     percentil = reglapercentil),
                .names = "out2_{.fn}"))

# Outliers detectados en Petal.Length2 por cada regla
iris_imp %>% summarise(across(starts_with("out2_"), ~ sum(.x, na.rm = TRUE)))
```

```
# A tibble: 1 x 4
  out2_sigma out2_hampel out2_boxplot out2_percentil
       <int>       <int>        <int>          <int>
1          2           3            3              8
```

- Representa los dos tipos de imputación empleados en un gráfico de dispersión (`Petal.Length1` vs. `Petal.Length2`) eliminando los outliers identificados con la regla `reglapercentil` y poniendo en diferente color los datos originales y los imputados.

```r
iris_imp %>%
  filter(!out2_percentil) %>%
  mutate(tipo = ifelse(imputado, "Imputado", "Original")) %>%
  ggplot(aes(x = Petal.Length1, y = Petal.Length2, color = tipo)) +
  geom_point(alpha = 0.8, size = 2) +
  scale_color_manual(values = c("Original" = "grey60", "Imputado" = "red")) +
  labs(title = "Imputación con la media global vs. media por especie",
       subtitle = "Sin outliers según la regla de percentiles",
       x = "Petal.Length1 (media global)",
       y = "Petal.Length2 (media por especie)", color = NULL) +
  theme_minimal()
```


Imputación con la media global vs. media por especie
Sin outliers según la regla de percentiles
Petal.Length2 (media por especie)
Petal.Length1 (media global)
7
6
5
4
3
2
2
3
4
5
6
Imputado
Original